SAFARI ZANGU VENTURES — INTERNAL REFERENCE

Booking System
Workflow Map

Every table in the database, every actor who touches the system, and the exact path a booking takes — through any of its three entry points — from first contact to a finished, fulfilled trip.

Document	**Booking Workflow Map**
Scope	**31 tables, 1 reporting view, 5 actor roles**
Generated	**19 October 2026**

ACTORS

GUEST	AGENT	STAFF	ADMIN	SUPERADMIN
No login. Books a package directly, submits a custom request, tracks a booking by ID + password, leaves a review.	Agent portal only. Scoped to their own leads, bookings, quotations and commissions — never another agent's.	Full operational access in the admin panel. No Settings or pricing-sensitive pages.	Everything Staff has, plus Settings, pricing rules and user management.	Same as Admin, plus bypasses the per-user status-change / delete locks on other accounts.

All five sign-in roles live in one table, `admin_users` — `role` is what separates them, not separate tables. Two more tables hold people who *cannot* sign in: `staff` (field guides/drivers) and `team_members` (public bios).

DATABASE TABLES BY DOMAIN

CATALOG — WHAT'S FOR SALE

TABLE	PURPOSE
`packages`	The marketed product — price, itinerary link, cost_breakdown (incl. Adults/Children Service Charge rates)
`itineraries`	Shared day-by-day template a package can link to instead of duplicating content
`accommodations, vehicles, activities, park_fees`	Raw cost components an itinerary day draws on
`accommodation_seasonal_rates, seasonal_rates`	Multipliers applied by travel month
`currency_rates`	Display-only FX conversion — all real charges stay in USD

SALES PIPELINE — TURNING INTEREST INTO A SALE

TABLE	PURPOSE
`leads`	An agent's prospect tracker — pipeline_stage from lead through booked/lost
`custom_safari_requests`	A bespoke enquiry (public, or agent-submitted via submitted_by_agent_id) — status new through converted/closed
`quotations`	The priced proposal — real itemized line_items, created_by for agent ownership, links back to its lead or request
`commissions`	Created automatically the moment a lead reaches 'booked'

BOOKING & MONEY — THE ACTUAL SALE

TABLE	PURPOSE
`bookings`	The authoritative record — who, what, when, total/deposit, payment_status, tracking_password_hash
`customers`	One deduped row per email, linked from bookings/quotations/custom_safari_requests
`booking_reminders`	The 4 scheduled lifecycle emails — deposit, balance, pre-departure, post-trip

FIELD OPERATIONS — DELIVERING THE TRIP

TABLE	PURPOSE
`booking_assignments`	Which guide (staff) and vehicle are dispatched to a confirmed booking
`incidents`	Anything that goes wrong on a trip, tied to a booking_ref
`suppliers`	Third-party partners — lodges, transfer companies

DATABASE TABLES BY DOMAIN (continued)

COMMUNICATIONS

TABLE	PURPOSE
`email_templates, email_queue`	Every outbound email is rendered from a template, then queued — so a mail outage never loses the message
`contact_messages`	The general "Contact Us" inbox

PEOPLE & REPUTATION

TABLE	PURPOSE
`team_members`	Public-facing "About Us" bios
`agent_targets`	Monthly KPI targets vs. actuals per agent
`reviews`	Post-trip guest feedback, optionally published

SYSTEM

TABLE	PURPOSE
`settings`	Key/value overrides behind Settings → General — deposit %, VAT, Service Charge, Maintenance Mode, etc.
`activity_log`	Audit trail of who did what
`analytics_events`	Raw site-traffic events
`schema_migrations`	Tracks which sql/*.sql files have been applied
`vw_package_profitability` (view)	Joins packages against operational_cost to compute margin live — nothing stored

How a finished booking really happens

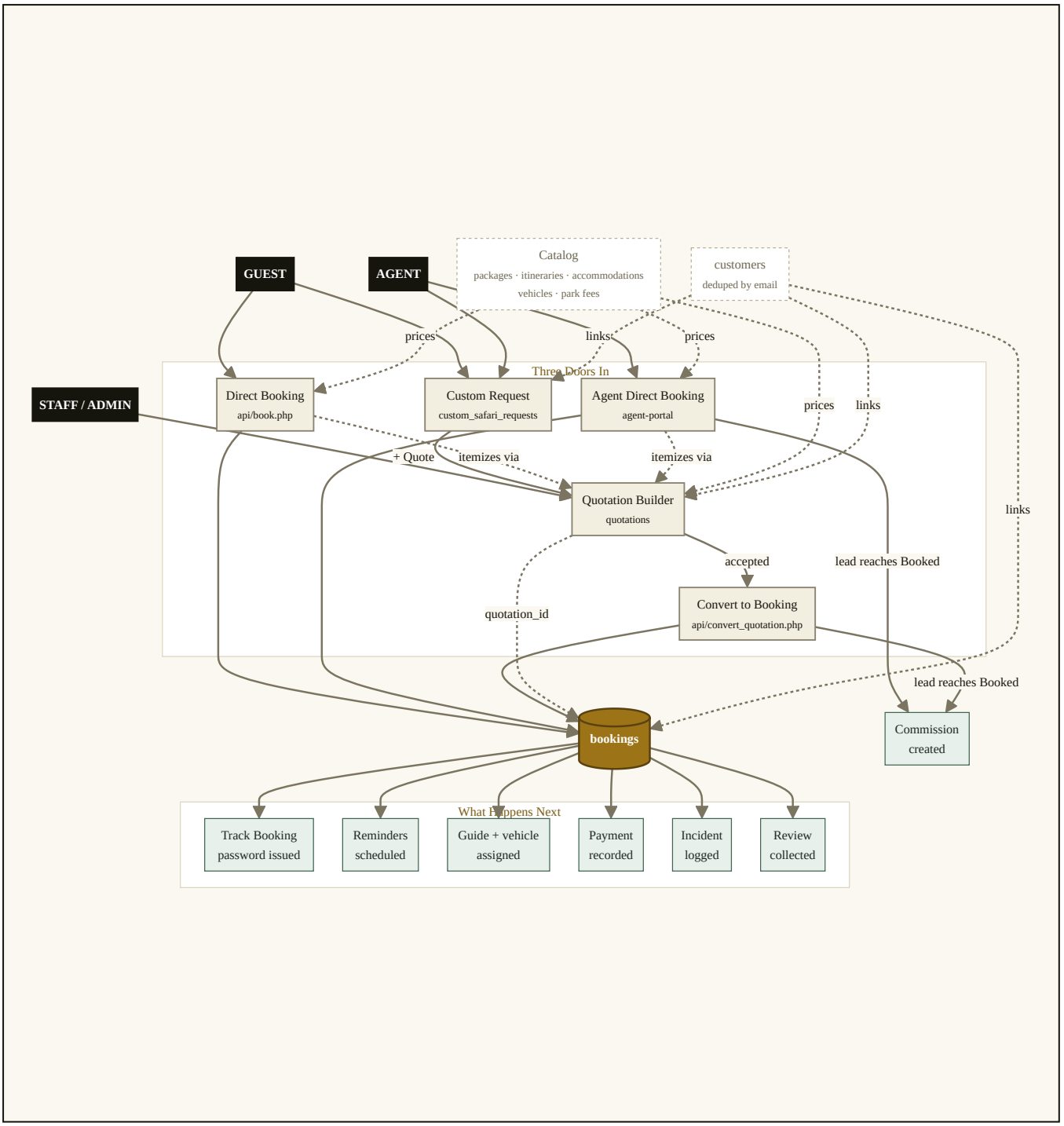
There are exactly three doors in, and all three converge on `bookings` and `quotations` from there on — see the diagram on the next page.

Door 1 — Guest books directly. The package is priced live (itemized if it has a linked itinerary, else a flat per-person rate); a `customers` row is found or created; `bookings` and a matching `quotations` row are inserted together; reminders are scheduled; a Track Booking password is issued and emailed.

Door 2 — Agent books for a client. Same pricing/booking/password/email steps as Door 1, plus the quotation is stamped with the agent's id for ownership, and a `leads` row is carried straight to `booked`, which creates the `commissions` row.

Door 3 — A request becomes a booking. An admin or agent converts an already-accepted quotation (itself built earlier from a custom request or a lead) into a real booking, at the exact numbers the client already saw. If it came from a lead or request, that record advances too.

WORKFLOW DIAGRAM



Actor
Who's doing something.

Entry / process
The door a booking comes through.

Core record
The one bookings row everything else reads from.

Lifecycle event
Fires automatically once a booking exists.

Supporting data
Reference tables, not a flow step.

> **One rule holds across all three doors:** ownership of a booking or a custom request is never stored on the record itself — it's *derived* from whichever quotation created it (`quotations.created_by`), with one deliberate exception for a custom request an agent submits before any quotation exists yet (`submitted_by_agent_id`). Commission only fires on the two doors that involve an agent.